Design and Analysis of Algorithms

Department of Computer Science and Engineering

ASSIGNMENT 4

Course Code & Title	24UCYPC303 – Design and Analysis of Algorithms
Unit	Unit IV – Backtracking and Branch-and-Bound
Targeted CO	COPC303.4
Maximum Marks	40
Date of Submission	24 October 2026
Name / PRN	Rushikesh Patil 2124UCSM1037

Q1. Backtracking and Branch-and-Bound: Principles and Control Abstractions

(a) Principle of backtracking and its control abstraction

Principle. Backtracking solves a problem whose answer can be written as a vector $(x_1, x_2, \ldots, x_n)$ by building that vector one component at a time, in a depth-first manner, over a **state space tree**. After choosing a value for x_n, a **bounding function** is applied to the partial vector $(x_1,\ldots,x_n)$. If the partial vector can no longer lead to a valid answer, the whole subtree below it is pruned and the algorithm *backtracks* to the previous level to try the next candidate value for x_{n-1}. Only promising nodes are expanded, so far fewer nodes are visited than in exhaustive search.

Control abstraction (recursive form):

```
Algorithm Backtrack(k)
// x[1..k-1] already hold the first k-1 components of the solution vector.
// T(x[1..k-1]) = set of all possible values for x[k] given x[1..k-1].
// B_k(x[1..k]) = bounding function (true if x[1..k] may lead to an
answer).
{ for each x[k] in T(x[1],...,x[k-1]) such that B_k(x[1],...,x[k]) = true
do
    { if (x[1],...,x[k]) is a path to an answer node then
        write (x[1:k]);
        if (k < n) then Backtrack(k+1);
    }
}
// Initial call: Backtrack(1)
```

Terminology

Term	Meaning	Example
Solution vector	The n-tuple $(x_1,\ldots,x_n)$ that represents one candidate solution; each x_i is chosen from a finite set.	4-queens: x_i = column of the queen in row i; sum of subsets: $x_i \in \{0,1\}$

Explicit constraints	Rules that restrict each x_i to a given set of values (independent of other components). They define the solution space.	$x_i \in \{1,\ldots,n\}$ for n-queens; $x_i \in \{0,1\}$ for subsets

Implicit constraints	Rules that state how the components must relate to one another for the vector to be a solution.	No two queens share a column or diagonal; $\Sigma\, w_i x_i = m$
Bounding function	A function $B_n(x_1..x_n)$ that returns false when the partial vector cannot be extended to a solution, so the subtree is killed.	Place(k,i) for n-queens; $s + w_n + \ldots > m$ for subset sum

(b) Principle and control abstraction of branch-and-bound; comparisons

Principle. Branch-and-bound (B&B) is used for optimisation problems. The state space tree is generated in a breadth-first/best-first manner: the **E-node** (node being expanded) stays the E-node until it is dead, and **all its children are generated at once**. Each generated node becomes a *live node* and is stored in a list of live nodes. For every node a bound $\hat{c}(x)$ on the best value reachable below it is computed; a node is **killed** if its bound cannot beat the best solution found so far (upper bound U for a minimisation problem, i.e. kill when $\hat{c}(x) > U$). The way the next E-node is picked from the live-node list defines the strategy (FIFO, LIFO, LC).

Control abstraction (least-cost search, LC):

```
Algorithm LCSearch(t)
// t = root of the state space tree; c^(x) = estimated cost of node x.
{ if t is an answer node then { output t; return; }
    E := t;                          // E-node
    initialize the list of live nodes to empty;
    repeat
    { for each child x of E do
        { if x is an answer node then { output path from x to t; return;
}
            Add(x);                  // x is added to the list of live
nodes parent(x) := E;
        }
        if there are no more live nodes then { write('no answer node');
return; }
        E := Least();                // delete and return the best live
node
    } until (false);
}
```

FIFO and LIFO search use the same skeleton; only **Add()** and **Least()** change (queue / stack / priority queue).

Backtracking vs. branch-and-bound

Basis	Backtracking	Branch-and-bound
Applied to	Decision / enumeration problems (find one or all solutions that satisfy constraints)	Optimisation problems (minimise or maximise an objective)
Tree traversal	Depth-first search (DFS)	BFS-like or best-first (FIFO, LIFO or LC)
Node expansion	One child at a time; current node remains active while children are tried	All children of the E-node are generated at once; E-node is then dead
Pruning rule	Bounding function from constraints (infeasible partial solution)	Bound on the cost (ĉ(x) compared with best value U), plus infeasibility
Memory	Only the current path (recursion stack), O(n)	Needs a list of live nodes, can be exponential
Order of solutions	Stops at the first or enumerates all	Stops when an answer node with least cost is chosen

Comparison of FIFO, LIFO and LC strategies

Strategy	Live-node data structure	Order of node expansion
FIFO B&B	Queue (First-In-First-Out)	Nodes are expanded in the order they were generated, i.e. level by level (breadth-first). The oldest live node becomes the next E-node.
LIFO B&B	Stack (Last-In-First-Out)	The most recently generated live node becomes the next E-node, so the tree is explored depth-first; the children of the newest node are expanded before older siblings.
LC B&B	Priority queue / min-heap keyed on ĉ(x)	The live node with the least estimated cost ĉ(x) is expanded next (best-first). Ties are broken in favour of the deeper / smaller-u node. It usually generates the fewest nodes because it heads straight for the most promising region.

Q2. N-Queens and Sum of Subsets using Backtracking

(a) 4-Queens problem

Let x[i] be the column of the queen placed in row i. Two queens at (j, x[j]) and (k, i) attack each other if they are in the same column (x[j] = i) or on the same diagonal (|x[j] − i| = |j − k|).

```
Algorithm Place(k, i)
// Returns true if a queen can be placed in row k, column i.
{ for j := 1 to k-1 do if ( x[j] = i or Abs(x[j] - i) =
    Abs(j - k) ) then return false;
    return true;
}

Algorithm NQueens(k, n)
{ for i := 1 to n do if
    Place(k, i) then
        { x[k] := i; if (k = n) then write
            (x[1:n]); else NQueens(k+1,
            n);
        }
}       // call NQueens(1, 4)
```

Working (which columns are rejected by Place):

Partial vector	Row	Column tried → result
()	1	1, 2, 3, 4 all accepted (explored in order)
(1)	2	1 ✗ same column; 2 ✗ diagonal; **3** ✓ ; **4** ✓
(1,3)	3	1 ✗ col; 2 ✗ diag; 3 ✗ col; 4 ✗ diag → dead end, backtrack
(1,4)	3	1 ✗ col; **2** ✓ ; 3 ✗ diag; 4 ✗ col
(1,4,2)	4	1 ✗ col; 2 ✗ col; 3 ✗ diag with row 3; 4 ✗ col → dead end
(2)	2	1 ✗ diag; 2 ✗ col; 3 ✗ diag; **4** ✓
(2,4)	3	**1** ✓ ; 2 ✗ col; 3 ✗ diag; 4 ✗ col
(2,4,1)	4	1 ✗ ; 2 ✗ diag; **3** ✓ ; 4 ✗ → **solution (2,4,1,3)**
(3)	2	**1** ✓ ; 2 ✗ diag; 3 ✗ col; 4 ✗ diag
(3,1)	3	1 ✗ ; 2 ✗ diag; 3 ✗ ; **4** ✓
(3,1,4)	4	1 ✗ ; **2** ✓ ; 3 ✗ ; 4 ✗ → **solution (3,1,4,2)**
(4)	2	**1** ✓ ; **2** ✓ ; 3 ✗ diag; 4 ✗ col

(4,1)	3	1 ✗; 2 ✗ diag; **3** ✓; 4 ✗
(4,1,3)	4	1 ✗; 2 ✗ diag; 3 ✗; 4 ✗ → dead end
(4,2)	3	1 ✗ diag; 2 ✗; 3 ✗ diag; 4 ✗ col → dead end

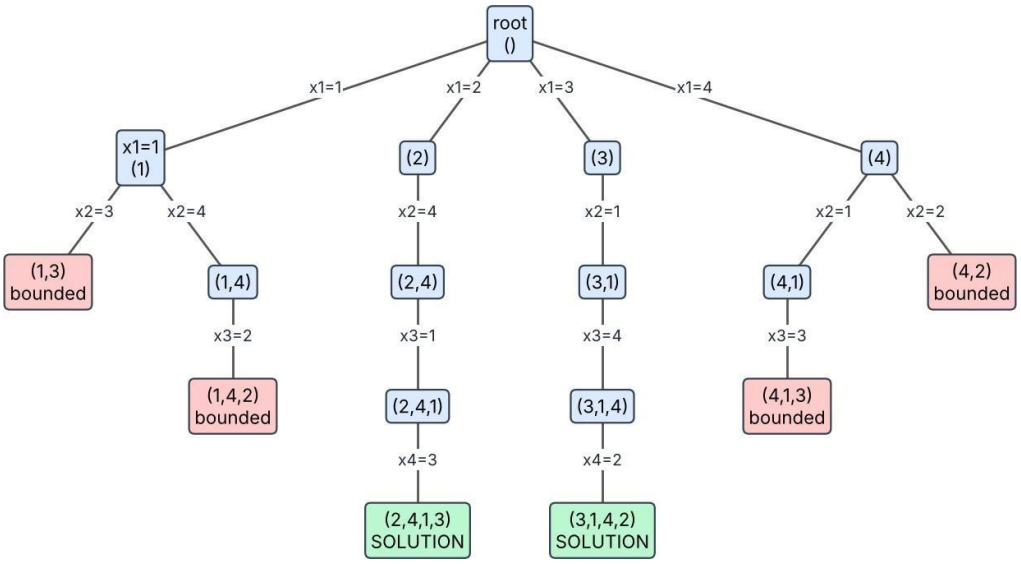


Figure 1: State space tree for 4-queens (red = bounded/dead nodes, green = solutions). Nodes are labelled by the partial solution vector.

Only **17 nodes** are generated in the pruned tree, against 341 nodes in the full tree ($1 + 4 + 4^2 + 4^3 + 4^4$).

Solution vectors: (2, 4, 1, 3) and (3, 1, 4, 2). These are mirror images of each other, so 4-queens has exactly 2 solutions.

Solution (2,4,1,3)	Solution (3,1,4,2)
Row1: . Q . . Row2: . . . Q Row3: Q . . . Row4: . . Q .	Row1: . . Q . Row2: Q . . . Row3: . . . Q Row4: . Q . .

(b) Sum of subsets: w = {7, 11, 13, 24}, m = 31

The weights are already sorted in non-decreasing order. Let $x_i = 1$ if w_i is included. At level k, with $s = \Sigma_{i<k} w_i x_i$ and $r = \Sigma_{i \geq k} w_i$ (remaining weight), a node is generated only if it is promising:

- **Left child ($x_k = 1$)** is generated if $s + w_n \leq m$ (and the answer is output if $s + w_n = m$, otherwise we require $s + w_n + w_{n+1} \leq m$).
- **Right child ($x_k = 0$)** is generated if $s + r - w_n \geq m$ and $s + w_{n+1} \leq m$.

```
Algorithm SumOfSub(s, k, r)
{
    x[k] := 1;                                    // left child
    if (s + w[k] = m) then write (x[1:k]);
    else if (s + w[k] + w[k+1] <= m) then SumOfSub(s + w[k], k+1, r -
w[k]);
    if ((s + r - w[k] >= m) and (s + w[k+1] <= m)) then   // right child
    {
        x[k] := 0;
        SumOfSub(s, k+1, r - w[k]);
    }

}     // initial call SumOfSub(0, 1, 55)   since 7+11+13+24 = 55
```

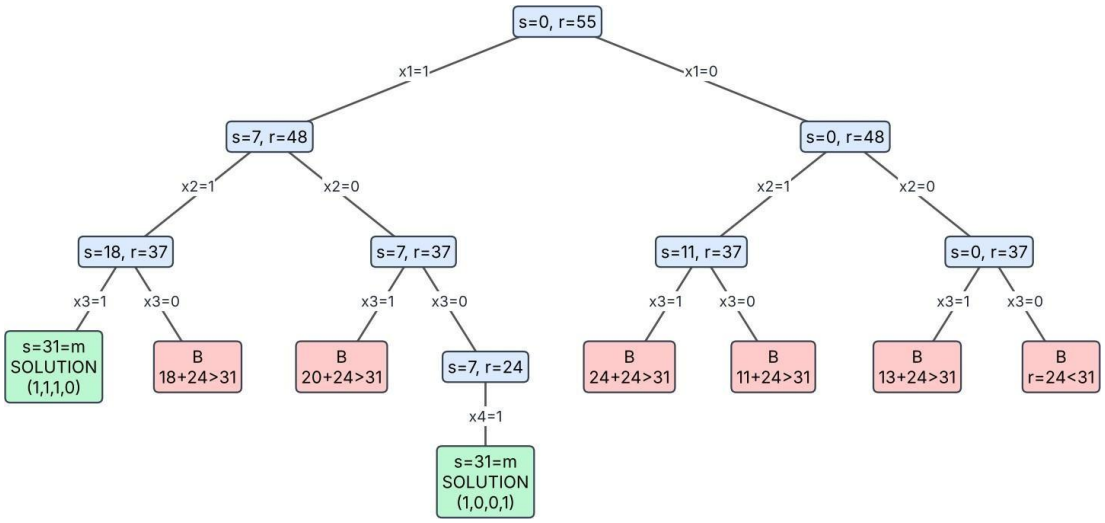


Figure 2: State space tree (each node shows s and r; B = bounded node with the failed condition; green = subset with sum 31).

Node-by-node working

Node (s, k, r)	Left child (include w_k)	Right child (exclude w_k)
(0, 1, 55)	7 ≤ 31 → (7, 2, 48)	0+55−7 = 48 ≥ 31 and 0+11 ≤ 31 → (0, 2, 48)
(7, 2, 48)	7+11 = 18; 18+13 = 31 ≤ 31 → (18, 3, 37)	7+48−11 = 44 ≥ 31, 7+13 ≤ 31 → (7, 3, 37)
(18, 3, 37)	18+13 = 31 = m → **solution (1,1,1,0)**	18+24−13 ≥ 31 but 18+24 = 42 > 31 → bounded
(7, 3, 37)	7+13 = 20; 20+24 = 44 > 31 → bounded	7+24 = 31 ≥ 31 and 7+24 ≤ 31 → (7, 4, 24)
(7, 4, 24)	7+24 = 31 = m → **solution (1,0,0,1)**	s + r − w_4 = 7 < 31 → not generated
(0, 2, 48)	11 ≤ 31, 11+13 ≤ 31 → (11, 3, 37)	0+37 ≥ 31, 0+13 ≤ 31 → (0, 3, 37)

(11, 3, 37)	11+13 = 24; 24+24 = 48 > 31 → bounded	11+24 = 35 > 31 → bounded
(0, 3, 37)	13; 13+24 = 37 > 31 → bounded	r = 24 < 31 → bounded

Subsets whose sum is 31:

- {7, 11, 13} → x = (1, 1, 1, 0) (7 + 11 + 13 = 31)
- {7, 24} → x = (1, 0, 0, 1) (7 + 24 = 31)

The bounding conditions prune 6 branches, so the search generates only 10 nodes (including the 2 solution nodes) instead of $2^5 - 1 = 31$ nodes.

Q3. Graph Colouring using Backtracking

Problem. Colour the vertices of a graph with at most m colours so that no two adjacent vertices get the same colour. The solution vector is $(x_1,\ldots,x_n)$, where x_i {1,…,m} is the colour of vertex i. G is given by its adjacency matrix G[1:n, 1:n].

```
Algorithm mColoring(k)
// x[1..k-1] are already coloured; G is the adjacency matrix.
{ repeat
    {
        NextValue(k);                // assign a legal colour to x[k]
          if (x[k] = 0) then return; // no new colour possible ->
backtrack  if  (k  =  n)  then  write
        (x[1:n]); else mColoring(k+1);
    } until (false);
}

Algorithm NextValue(k)
// x[1..k-1] is coloured legally. On exit x[k] = next legal colour // (in
the range 1..m) not used by an adjacent vertex, or 0 if none exists.
{ repeat
    { x[k] := (x[k] + 1) mod (m + 1);    // next highest colour if (x[k] =
        0) then return;      // all colours exhausted for j := 1 to n do
        if ( G[k, j] != 0 and x[k] = x[j] ) then break; // adjacent,
same colour
        if (j = n + 1) then return;              // new colour found
    } until (false);
}     // initial call mColoring(1) with x[] initialised to 0
```

Given graph: V = {1,2,3,4,5}, E = {(1,2), (1,3), (2,3), (2,4), (3,4), (4,5)}, m = 3.

Adjacent vertices: 1 → {2,3}; 2 → {1,3,4}; 3 → {1,2,4}; 4 → {2,3,5}; 5 → {4}.

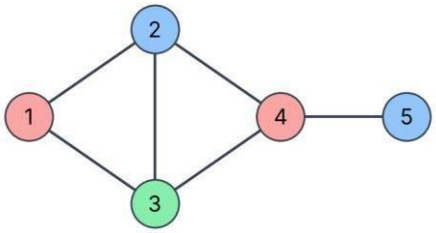


Figure 3: The graph with the colouring (1,2,3,1,2) (colours red = 1, blue = 2, green = 3).

Working (first solution). $x_1 = 1$. For vertex 2, colour 1 is rejected (adjacent to 1), so $x_2 = 2$. For vertex 3, colours 1 and 2 are rejected (adjacent to 1 and 2), so $x_3 = 3$. For vertex 4 (adjacent to 2 and 3), colours 1 is the first free one, $x_4 = 1$ (colours 2 and 3 are clashes). Vertex 5 is adjacent only to 4, so colour 1 is rejected and $x_5 = 2$ is accepted, giving (1,2,3,1,2). Backtracking then tries $x_5 = 3$, giving (1,2,3,1,3), and so on.

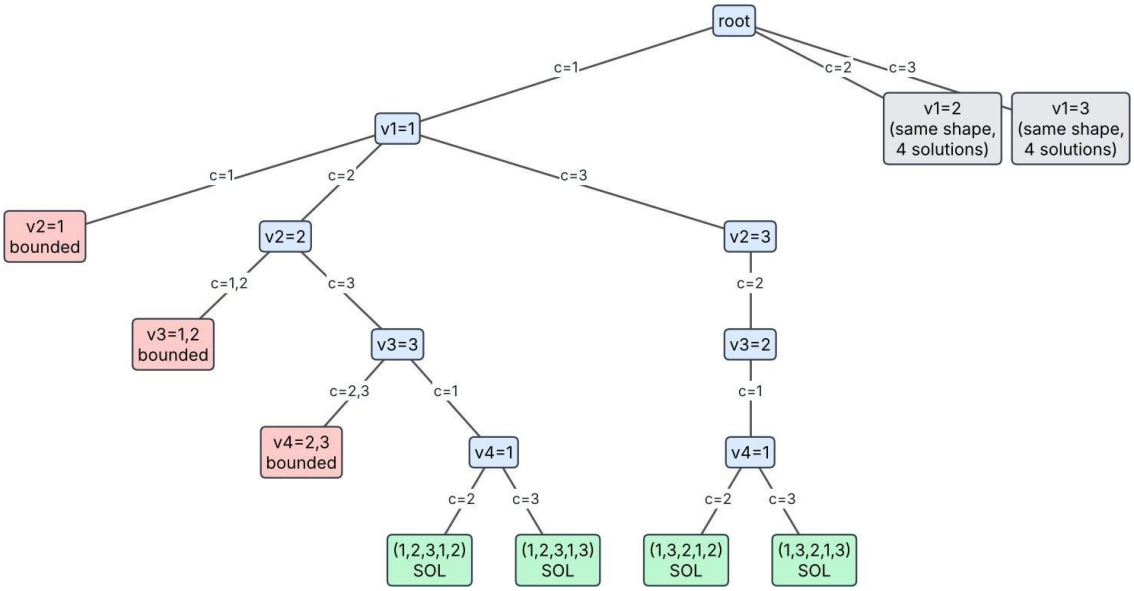


Figure 4: State space tree (level i = vertex i). The subtree under $v_1 = 1$ is shown in full; the subtrees under $v_1 = 2$ and $v_1 = 3$ have identical shape (colours are just permuted).

Valid 3-colourings There are exactly 12:

#	Colouring	#	Colouring	#	Colouring
1	(1,2,3,1,2)	5	(2,1,3,2,1)	9	(3,1,2,3,1)
2	(1,2,3,1,3)	6	(2,1,3,2,3)	10	(3,1,2,3,2)
3	(1,3,2,1,2)	7	(2,3,1,2,1)	11	(3,2,1,3,1)
4	(1,3,2,1,3)	8	(2,3,1,2,3)	12	(3,2,1,3,2)

Counting check: vertex 1 has 3 choices, vertex 2 has 2, vertex 3 has 1 (it is adjacent to both 1 and 2), vertex 4 has 1 (adjacent to 2 and 3, so it must reuse the colour of vertex 1) and vertex 5 has 2: $3 \times 2 \times 1 \times 1 \times 2 = 12$.

Chromatic number. Vertices 1, 2, 3 form a triangle (K_3), so at least 3 colours are needed; a 3-colouring exists (above), and the backtracking run with m = 2 fails at vertex 3 for every choice. Hence .

Worst-case analysis. The state space tree has n + 1 levels and every internal node has m children. The total number of nodes is

$$1 + m + m^2 + \ldots + m^n = (m^{n+1} - 1)/(m - 1).$$

For n = 5, m = 3 this is $(3^6 - 1)/2$ = **364 nodes**. The nodes at levels 0 … n−1 (121 of them) are internal and each performs an O(n) check of adjacency for each of its m children, so the worst-case running time is $O(n \cdot m^n)$ (here $\approx 5 \times 3^5 = 1215$ elementary checks). In practice the bounding function prunes heavily: the actual pruned tree for this graph has only 1 + 3 × 11 = **34 nodes**.

Q4. Branch-and-Bound: 0/1 Knapsack and Travelling Salesperson Problem

(a) 0/1 knapsack using FIFO branch-and-bound

Instance: n = 4, P = (10, 10, 12, 18), W = (2, 4, 6, 9), M = 15. The items are already in non-increasing order of P_i/W_i = 5, 2.5, 2, 2.

Knapsack is a maximisation problem, so it is converted into a minimisation problem by using negative profits. For a node x (decisions on $x_1..x_k$ made, profit p, weight w):

- **ĉ(x) (cost / lower bound)** = −(p + profit from filling the remaining capacity greedily, allowing a fraction of the last item).
- **u(x) (upper bound on the minimum cost)** = −(p + profit from adding only whole items that fit); this is the value of a real feasible solution.
- U = the smallest u(x) seen so far (initially u(root)). A node is **killed** if ĉ(x) > U. Infeasible nodes (w > M) are killed immediately.

Root: items 1, 2, 3 fit (w = 12, p = 32) and 3/9 of item 4 gives 6 more: ĉ = −38; whole items only: u = −32. So U = −32. The nodes are expanded in FIFO (queue) order, the x = 1 child being generated before the x = 0 child.

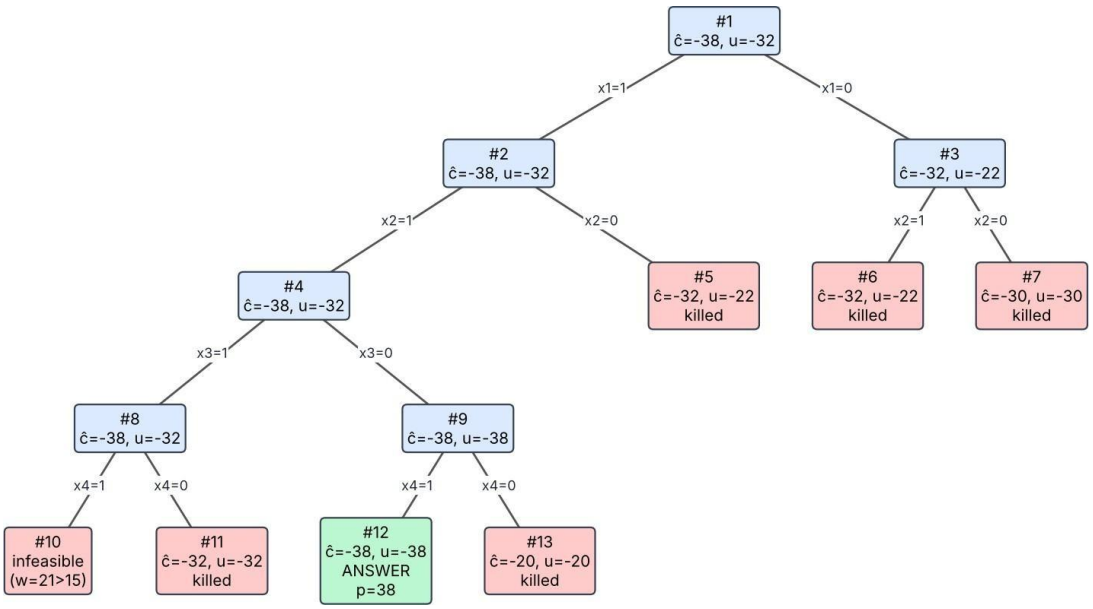


Figure 5: FIFO branch-and-bound state space tree (#n = order of generation; ĉ = cost, u = upper bound; red = killed/infeasible, green = answer node).

Node	Decisions	p	w	ĉ	u	Status
#1	root	0	0	−38	−32	Expanded; U = −32
#2	1 ? ? ?	10	2	−38	−32	Expanded
#3	0 ? ? ?	0	0	−32	−22	Live (ĉ = U, not > U); expanded
#4	1 1 ? ?	20	6	−38	−32	Expanded
#5	1 0 ? ?	10	2	−32	−22	Live when generated; killed once U = −38
#6	0 1 ? ?	10	4	−32	−22	Live when generated; killed once U = −38
#7	0 0 ? ?	0	0	−30	−30	ĉ = −30 > U = −32 → killed
#8	1 1 1 ?	32	12	−38	−32	Expanded
#9	1 1 0 ?	20	6	−38	−38	Expanded; U becomes −38
#10	1 1 1 1	41	21	–	–	Infeasible (w = 21 > 15)
#11	1 1 1 0	32	12	−32	−32	−32 > U = −38 → killed
#12	1 1 0 1	38	15	−38	−38	**Answer node**
#13	1 1 0 0	20	6	−20	−20	Killed

Result: the optimal solution is x = (1, 1, 0, 1) with weight 2 + 4 + 9 = 15 ≤ 15 and maximum profit 10 + 10 + 18 = **38**. (Verified by checking all 16 subsets.)

FIFO B&B generated **13 nodes** (6 of them expanded as E-nodes: #1, #2, #3, #4, #8, #9).

Comparison with LC branch-and-bound. LC always expands the live node with the least ĉ (ties broken by smaller u). The first expansions are the same (#1 → #2 → #4), but after node 7 ($x_3 = 0$, ĉ = u = −38) is generated, U = −38 and every live node with ĉ = −32 is killed immediately. Node 7 is then chosen before node 6 and its child ($x_4 = 1$) is the answer node, so the search stops without ever expanding the other live node.

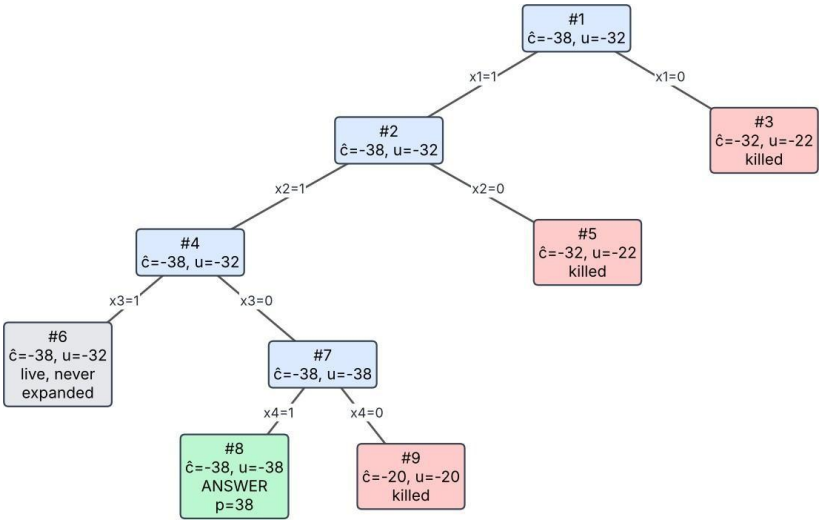


Figure 6: LC branch-and-bound tree for the same instance (9 nodes).

Strategy	Nodes generated	Nodes expanded	Answer
FIFO B&B	13	6	(1,1,0,1), profit 38
LC B&B	9	4	(1,1,0,1), profit 38

LC B&B generates 4 fewer nodes (about 31% fewer) because it follows the most promising node instead of exploring level by level; FIFO wastes effort on nodes (#3 and its subtree #6, #7, and node #5) that lie at shallow levels with poor bounds.

(b) Travelling salesperson problem least-cost branch-and-bound

Cost matrix (cities 1–4):

	1	2	3	4
1	∞	10	15	20
2	10	∞	35	25
3	15	35	∞	30
4	20	25	30	∞

Step 1 Row reduction. Subtract the minimum of each row from that row.

Row minima: 10, 10, 15, 20 → row reduction total = 10 + 10 + 15 + 20 = **55**.

	1	2	3	4
1	∞	0	5	10
2	0	∞	25	15
3	0	20	∞	15
4	0	5	10	∞

Step 2 Column reduction. Column minima of the row-reduced matrix: column 1 = 0, column 2 = 0, column 3 = 5, column 4 = 10 → column reduction total = 0 + 0 + 5 + 10 = **15**.

	1	2	3	4
1	∞	0	0	0
2	0	∞	20	5
3	0	20	∞	5
4	0	5	5	∞

Lower bound at the root: ĉ(root) = 55 + 15 = **70**. Every tour must cost at least 70.

Step 3 Bounds of the children of the root. Let A be the reduced matrix of the parent (above). For a child node corresponding to the edge (i, j):

Set all entries of row i and column j to ∞ (city i is left, city j is entered).
Set A(j, 1) = ∞ so that the partial path cannot close the tour prematurely.

Reduce the resulting matrix; let r be the total reduction.

(child) = (parent) + A(i, j) + r.

Child 2 edge (1,2): A(1,2) = 0. After blocking row 1, column 2 and A(2,1), the row minima are 5 (row 2), 0, 0 (r = 5); the column minima are 0, 5 (column 3), 0 (r = 5), so r = 10 and **= 70 + 0 + 10 = 80**.

Child 3 edge (1,3): A(1,3) = 0. Blocking row 1, column 3 and A(3,1): row minima 0, 5 (row 3), 0 → 5; column minima 0, 5 (column 2), 0 → 5; r = 10 and **= 70 + 0 + 10 = 80**.

Child 4 edge (1,4): A(1,4) = 0. Blocking row 1, column 4 and A(4,1): row minima 0, 0, 5 (row 4) → 5; all column minima are 0; r = 5 and **= 70 + 0 + 5 = 75**.

Child	Edge	A(i,j)	Reduction r	Lower bound
2	(1,2)	0	5 + 5 = 10	70 + 0 + 10 = 80
3	(1,3)	0	5 + 5 = 10	70 + 0 + 10 = 80
4	(1,4)	0	5 + 0 = 5	70 + 0 + 5 = 75

Reduced matrices of the children (after blocking and reduction):

Node 2

	1	2	3	4
1	∞	∞	∞	∞
2	∞	∞	10	0
3	0	∞	∞	5
4	0	∞	0	∞

Node 3

	1	2	3	4
1	∞	∞	∞	∞
2	0	∞	∞	5
3	∞	10	∞	0
4	0	0	∞	∞

Node 4

	1	2	3	4
1	∞	∞	∞	∞
2	0	∞	20	∞
3	0	20	∞	∞
4	∞	0	0	∞

LC search continues. The least-cost live node is node 4 (75), so it is expanded next: its children (1→4→2) and (1→4→3) both have bound 95. The nodes with bound 80 are now the cheapest: expanding node 2 gives (1→2→3) with bound 95 and (1→2→4) with bound 80, which leads to the complete tour 1 → 2 → 4 → 3 → 1 with cost 10 + 25 + 30 + 15 = **80**. Node 3 leads to the same tour in the reverse direction (1 → 3 → 4 → 2 → 1, cost 80). Every other live node has a bound ≥ 80, so the optimum tour cost is **80**; the lower bound 70 at the root is therefore not attained, which is normal for a reduction bound.

Check by enumeration: 1-2-3-4-1 = 95; 1-2-4-3-1 = **80**; 1-3-2-4-1 = 95; 1-3-4-2-1 = **80**; 1-4-2-3-1 = 95; 1-4-3-2-1 = 95.

End of Assignment